Unlearnable, or Unmeasured? On the Reliability of Difficulty Labels in RLVR

Chandak Chakma[1] **Syed Nazmus Sakib**[1] **Nafiul Haque**[1] **Shifat E. Arman**[1,2,*]

[1]Department of Robotics and Mechatronics Engineering, University of Dhaka, Bangladesh
[2]Department of Computer Science, University of Oxford, UK

[*]Work done at University of Dhaka. Corresponding author: shifatearman@du.ac.bd

Abstract

Reinforcement learning with verifiable rewards (RLVR) has become an important approach for improving reasoning during post-training. Recent work suggests that some difficult prompts remain resistant to learning even when they occasionally produce correct solutions. We revisit this unlearnability phenomenon and find that the affected prompts do improve, at roughly one third of the learnable rate, while the difficulty-defined set used to study them is much less reproducible than expected. These difficulty labels are estimated from a limited number of sampled responses. Combining them across seeds can further change which prompts are selected instead of simply reducing measurement noise. We develop a sampling-based framework for quantifying this instability and determining how much evaluation is required for difficulty assignments to reproduce reliably. We also revisit the gradient-similarity evidence proposed to explain unlearnability and show that part of the observed separation arises because difficult prompts provide fewer correct rollouts from which their gradients can be estimated. Matching this sample count weakens the gradient difference but does not remove it. Overall, the slow-learning phenomenon survives our reanalysis, while both the prompts used to define it and the evidence used to explain it require more careful measurement.

Code: github.com/syed-nazmus-sakib/Unlearnable-RLVR
Project page: syed-nazmus-sakib.github.io/Unlearnable-RLVR

1 Introduction

Reinforcement learning with verifiable rewards (RLVR) has become one of the most effective approaches for improving reasoning in post-training. RLVR has produced substantial gains in mathematical and programmatic reasoning and now forms a central component of post-training for modern reasoning models (Lambert et al., 2024, Shao et al., 2024, Guo et al., 2025, Yu et al., 2025). At the same time, its success has exposed a more fundamental question: what determines whether a reasoning problem can actually be learned through RLVR? Some studies suggest that RLVR can push models toward stronger reasoning behavior, while others find that its gains remain closely tied to capabilities already present in the base policy (Yue et al., 2025, Wen et al., 2026). Understanding where learning succeeds, where it slows down, and why has therefore become a central problem in post-training.

Prompt difficulty has emerged as one way to study and control this phenomenon. Recent methods use a model's empirical success rate to decide which prompts should be replayed, filtered, routed, reweighted, or given additional training effort (Baroian and Berger, 2026, Luo et al., 2026, Li et al., 2026, Pang et al., 2026, Qu et al., 2026, Bae et al., 2026, Tang et al., 2026). Difficulty is also used

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Workshop: Transitioning from Pre-Training to Post-Training.

to probe the limits of RLVR itself. Most strikingly, Chen et al. (2026) identified a group of hard prompts that occasionally produce successful trajectories yet improve very little during training. They describe these prompts as *unlearnable* and connect their behavior to lower within-group gradient similarity, suggesting a possible representation failure. If correct trajectories are already available but the model still fails to learn from them, the limitation would be deeper than ordinary exploration or reward sparsity.

Both difficulty-aware training and claims about unlearnability depend on how reliably hard prompts can be identified. Difficulty is estimated from sampled responses, so the same prompt can cross a threshold across repeated evaluations even when the model is unchanged. Aggregating these noisy labels can further change which prompts are selected rather than simply reduce measurement noise. We revisit the unlearnability result of Chen et al. (2026) from this perspective. The reported prompts still improve much more slowly than the learnable group, at roughly one third of the rate, but the set itself is highly sensitive to construction. Using the same candidate prompts, the published aggregation rule, applied across our five runs, returns 74 prompts, while an alternative aggregation returns 228; individual seeds remain between 145 and 150. A same-checkpoint evaluation reproduces much of this disagreement without any change in training. The slow-learning separation therefore appears in our data, but the prompts used to define it are much less stable than the label suggests.

This raises a broader question: how much evaluation is needed for a difficulty label to be reliable? We show that reproducibility depends on the rollout budget and on how many prompts lie near the threshold. In our setting, 8 to 16 rollouts per prompt yield only 0.63 to 0.73 agreement, while 0.90 agreement requires a projected 242 rollouts. We also revisit the gradient-similarity explanation and find a separate measurement issue: hard prompts provide fewer correct rollouts, so their gradients are estimated from fewer samples. Matching this sample count reduces the easy-to-unlearnable similarity ratio from 2.327 times to 1.539 times, although a clear gap remains. Overall, the slow-learning phenomenon persists, but both the prompts used to define it and the evidence used to explain it depend strongly on measurement choices.

Our work makes three main contributions:

1. **We revisit what "unlearnable" means in RLVR.** The reported prompts do improve during training, but at roughly one third of the learnable rate, supporting a persistent slow-learning effect rather than literal non-learning. At the same time, we show that the set used to define this effect is highly sensitive to its construction: changing the aggregation rule moves the set from 74 to 228 prompts, while much of the apparent disagreement across training seeds can be reproduced by resampling the same checkpoint.
2. **We turn difficulty assignment into an explicit sampling-budget problem.** We characterize how reliably threshold-based difficulty labels can be reproduced as the number of rollouts increases, and use this relationship to determine the evaluation budget required for a chosen level of agreement. The resulting predictions match retrospective measurements and two prospectively frozen evaluations, providing a practical way to choose rollout budgets rather than setting them heuristically.
3. **We reassess the proposed gradient explanation for unlearnability.** Hard prompts provide fewer correct rollouts, which makes their prompt-level gradients less precisely estimated. Matching the number of correct rollouts reduces the easy-to-unlearnable gradient-similarity ratio from 2.327 times to 1.539 times. A substantial difference remains, so the original evidence is weakened rather than eliminated, and the mechanism behind the slow-learning effect remains unresolved.

2 Related work

Our work connects four lines of research in RLVR: understanding the limits of post-training, using prompt difficulty to guide training, explaining why some hard prompts remain difficult to learn, and studying the reliability of the measurements used to support these conclusions. We organize the related work around these four themes.

RLVR and the limits of post-training. Reinforcement learning with verifiable rewards has become an important approach for improving reasoning after pre-training, building on the use of

automatically checkable answers to supervise mathematical reasoning (Cobbe et al., 2021). Tülu 3 established a practical open recipe for reinforcement learning with automatically verifiable rewards, while DeepSeek-R1 demonstrated that large-scale reinforcement learning can substantially improve reasoning behavior (Lambert et al., 2024, Guo et al., 2025). Recent systems such as DAPO have further studied how sampling and optimization choices affect RLVR training (Yu et al., 2025). At the same time, several works have questioned what this training can actually teach. Yue et al. (2025) argue that RLVR may primarily sharpen capabilities already present in the base model rather than create entirely new reasoning capacity. Other work reports that prolonged reinforcement learning can expand the reasoning boundary beyond what is observed under shorter post-training runs (Liu et al., 2026). Chen et al. (2026) identify a related phenomenon at the prompt level: some difficult examples occasionally produce positive reward but improve very little during training. They describe these prompts as *unlearnable* and connect their behavior to differences in gradient similarity. Our work starts from this prompt-level observation, but asks whether the set of prompts used to support such a trainability claim is itself measured reliably.

Difficulty as a training signal. Prompt difficulty is increasingly treated as an operational quantity rather than only a descriptive property. Recent methods use estimated success rates to decide which prompts should be replayed, routed, reweighted, or prioritized. Prompt Replay focuses training on high-signal prompts using on-policy reuse (Baroian and Berger, 2026). DRIFT routes examples according to difficulty and training dynamics (Luo et al., 2026), while difficulty-aware group normalization changes the treatment of examples from different difficulty bands (Li et al., 2026). Other work uses rare-event statistics to allocate training effort (Pang et al., 2026), or predicts prompt difficulty with a smaller model before expensive sampling is performed (Qu et al., 2026). Off-context GRPO instead provides privileged information when hard prompts fail to produce useful rollouts (Agrawal et al., 2026). Recent methods also use curriculum construction and explicitly difficulty-aware optimization to decide which prompts receive training effort (Gao et al., 2026, Dai et al., 2026). Related methods use risk-sensitive exploration or influence-based data selection to improve the efficiency of RLVR (Jiang et al., 2026, Zhu et al., 2026). Recent extreme data-efficiency results further show that post-training outcomes can depend strongly on which training examples are selected (Wang et al., 2025b). These methods differ substantially in their objectives, but they share an important dependency: prompt difficulty is inferred from a finite number of sampled outcomes. We study the statistical reliability of that underlying measurement.

Why hard prompts fail to learn. A second line of work asks how difficult examples can be made learnable and what distinguishes successful from unsuccessful training signal. Process supervision evaluates intermediate reasoning rather than only the final answer (Uesato et al., 2022, Lightman et al., 2024), and recent RLVR methods combine process and outcome information during training (Ye et al., 2025). Other methods provide partial solution prefixes to increase the probability that hard prompts produce informative trajectories (Beliaev, 2026). Chen et al. (2026) instead study the geometry of per-prompt gradients and report lower gradient alignment for their unlearnable set, interpreting this as evidence of a representation failure. Gradient-based analyses have a broader history as tools for studying the influence of individual training examples and optimization behavior (Koh and Liang, 2017, Pruthi et al., 2020). Recent work has also studied optimization bias, entropy dynamics, token-level learning signal, credit assignment, and gradient utilization as possible determinants of RLVR learning behavior (Liu et al., 2025, Cui et al., 2025, Wang et al., 2025a, Hao et al., 2026, Kazemnejad et al., 2025, Fu et al., 2026). We do not propose a new mechanism for hard-prompt learning. Instead, we show that the gradient comparison itself depends strongly on how many successful rollouts are averaged for each prompt. Matching this sample count substantially reduces the reported separation, while leaving a residual difference that still requires explanation.

Measurement and reproducibility. Reproducibility work in reinforcement learning and machine learning has shown that seeds, evaluation protocols, benchmark variance, and search budgets can materially change reported conclusions (Henderson et al., 2018, Dodge et al., 2019, Agarwal et al., 2021, Bouthillier et al., 2021). Our setting differs in one important respect. We are not primarily asking whether an aggregate benchmark score reproduces. We study whether the *membership of a prompt-level set* reproduces when that membership is defined by thresholding a sampled success rate. This distinction matters because difficulty-aware post-training methods increasingly act on such sets directly. To our knowledge, prior RLVR work has not treated the aggregation rule and rollout budget behind difficulty assignment as part of the measurement procedure whose reliability must

itself be established. Our contribution is therefore complementary to both post-training research and the broader reproducibility literature: we characterize when a prompt-level trainability label is statistically stable enough to support downstream scientific claims.

3 Experimental Results

3.1 Experimental setup

We study the unlearnability phenomenon reported by Chen et al. (2026). Our primary experiments use Qwen2.5-0.5B (Yang et al., 2024) trained with GRPO (Shao et al., 2024) on 1,023 prompts from MATH (Hendrycks et al., 2021). We train five independent seeds with $G = 8$ rollouts per prompt and use the original verifier without modification. We retain the published easy, learnable, and unlearnable cohorts and the original difficulty threshold $\tau = 0.1$. We write D_u for the population produced by the published unlearnability construction.

The experiments below separate three sources of variation that are easy to conflate: variation from training, variation from finite evaluation, and variation introduced by the rule used to aggregate prompt-level decisions. Unless stated otherwise, set-membership analyses use $N = 128$ evaluation rollouts per prompt and compare independently constructed sets using Jaccard similarity. Gradient analyses use $N = 200$ initial-policy rollouts and form each prompt-level gradient by averaging over its correct rollouts. Confidence intervals are obtained by bootstrap with prompts resampled at the level appropriate to each statistic.

Our implementation is not byte-for-byte identical to the original training configuration. The most important difference is generation length: our training and primary evaluation use a 1,024-token cap, whereas the original scripts default to 5,120. This matters more for hard prompts than for easy ones, so we test evaluation-side sensitivity to the longer cap directly and avoid transferring training-dependent magnitudes beyond our configuration. Appendix E reports the full constant-by-constant audit and all known differences. Upon acceptance, we will release code, rollout artifacts, the exact aggregation rules, and timestamped records of the two frozen predictions.

3.2 The slow-learning separation appears, but literal non-learning does not

Do the prompts labeled unlearnable actually fail to learn? We first ask whether the reported unlearnable prompts actually show no improvement during RLVR training. The three published groups remain clearly separated throughout training (Figure 1), but the D_u group is not flat: reward increases by $+0.0422\,[+0.0259, +0.0590]$ per 100 optimizer steps, compared with $+0.1119\,[+0.0879, +0.1368]$ for learnable and $+0.3800\,[+0.3599, +0.4006]$ for easy prompts (and remains positive across all five seeds: mean $+0.0328$, range $+0.0266$ to $+0.0422$). Thus, the prompts labeled unlearnable improve at roughly one third of the learnable rate rather than not at all.

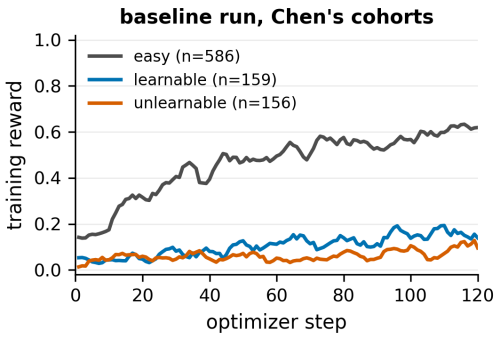


Figure 1: **The separation is present; the flatness is not.** Training reward for the published easy, learnable, and unlearnable cohorts. Slopes per 100 optimizer steps are $+0.3800$, $+0.1119$, and $+0.0422\,[+0.0259, +0.0590]$, respectively. The affected cohort remains markedly slower to improve, but its slope is not zero.

We use a within-prompt slope rather than a narrow terminal window to avoid finite-window sampling bias. Rewards are logged for every sampled group of $G = 8$ rollouts *prior* to GRPO's zero-variance update filter (including all $0/8$ and $8/8$ groups), whereas a 10-step terminal window (steps 111–120) visits only 60 of the 156 D_u prompts because each step draws only a minibatch. Fitting a linear trend within each prompt across all of its appearances over steps 1–120 (≈ 6.7 per prompt) and averaging across the fixed cohort retains all 156 D_u, 159 learnable, and 586 easy prompts with zero prompts dropped (Appendix A.1). As an independent check that does not rely on training-time logging, fixed-checkpoint evaluations of the same 156 D_u prompts from the initial policy to step

120 ($N = 256$) show positive pass-rate gains in every seed ($+0.1232$ to $+0.1306$). Throughout this paper, "unlearnable" and D_u therefore refer to the published label of Chen et al. (2026), not to literal non-learning.

3.3 The published construction is not a consistent estimator of a fixed-threshold set

Does the aggregation rule matter more than the training seed? Because these prompts improve slowly rather than not at all, a natural question is how stable the prompt set used to study them actually is. We separate variation across training seeds from variation caused by the rule used to combine seed-level labels (whereas Chen et al. (2026) combined three seeds, we train five to trace set size across $j = 1, \ldots, 5$ in Figure 2, top, where $j = 3$ averages 61.2 prompts, and apply their rule across all five runs). To compare aggregation rules fairly, we first evaluate all 1,023 prompts 128 times on the same reference checkpoint and fix the 292 candidate prompts with pass rate $\leq \tau = 0.1$. Applying the published rule across all five training runs leaves only **74** prompts in D_u, because a prompt is excluded if it is marked as `no_reward` in any of the five runs. If we change only this aggregation step and exclude a prompt only when it is marked as `no_reward` in all five runs, **228** prompts remain. In comparison, constructing D_u from each training seed separately gives much more similar set sizes, ranging from 145 to 150 prompts, with a mean of 147.8. A separate rule based only on the 128-response evaluation, where a prompt is selected if it receives zero correct responses, identifies 181 prompts. The final size of D_u therefore changes far more because of how the five runs are combined than because of which training seed is used (Figure 2).

Figure 2 also shows that this is not simply a disagreement between two particular constructions. As additional seeds are combined, the selected set changes systematically rather than settling toward a stable collection of prompts. Many prompts are flagged by some seeds but not others, and most of this disagreement is concentrated near the threshold $\tau = 0.1$, where only a small change in the number of successful responses can move a prompt from one side of the threshold to the other. The instability is therefore not spread uniformly across prompts; it is concentrated exactly where the difficulty decision is most sensitive to finite sampling. This suggests that combining more seeds is not simply reducing noise. The aggregation rule is also changing which prompts remain in the final set. That leads to the next question: how much of the disagreement we observe is actually caused by training different models, and how much would still appear if we evaluated the same model again with a different set of sampled responses?

What fails: the latent quantity, or the rule used to estimate it? The underlying notion of difficulty is not the problem. If $p(x)$ denotes the true probability that a fixed policy solves prompt x, then

$$D_\tau = \{x : p(x) \leq \tau\}$$

defines a clear target set: all prompts whose true success probability is at or below the chosen threshold τ. The difficulty comes from how this set is estimated in practice. The published procedure observes only a limited number of responses for each prompt, uses those responses to decide whether the prompt falls below the threshold in each run, and then combines these yes-or-no decisions across multiple runs. Combining the decisions in this way does more than reduce sampling noise; it gradually changes which prompts can remain in the set. In particular, if a prompt must be classified as difficult in every independent run, then as more runs are added, only prompts that are almost guaranteed to receive that label can survive. Under the binomial threshold model, this eventually pushes the retained set toward prompts with $p(x) = 0$. The subsequent `no_reward` exclusion then removes exactly these prompts, because they never produce a rewarded response. The procedure therefore does not approach the fixed set D_τ as more runs are added. It approaches a degenerate outcome created by the aggregation rule itself. This inconsistency concerns the regime in which each run supplies a finite, independently thresholded label and the number of such runs increases; it is not a claim that every procedure using repeated evaluation is inconsistent, since pooling the same rollouts into a single deeper estimate provides a direct finite-sample estimate of D_τ whose sampling error decreases with rollout budget. The exclusion creates a second problem: receiving no reward in the observed training runs is not the same as being unsolvable. Of the 218 prompts removed by the `no_reward` criterion, 116 are solved at least once in a later $N = 128$ evaluation. We therefore distinguish between prompts for which reward was not observed during training and prompts that the policy cannot solve, rather than treating the two as equivalent.

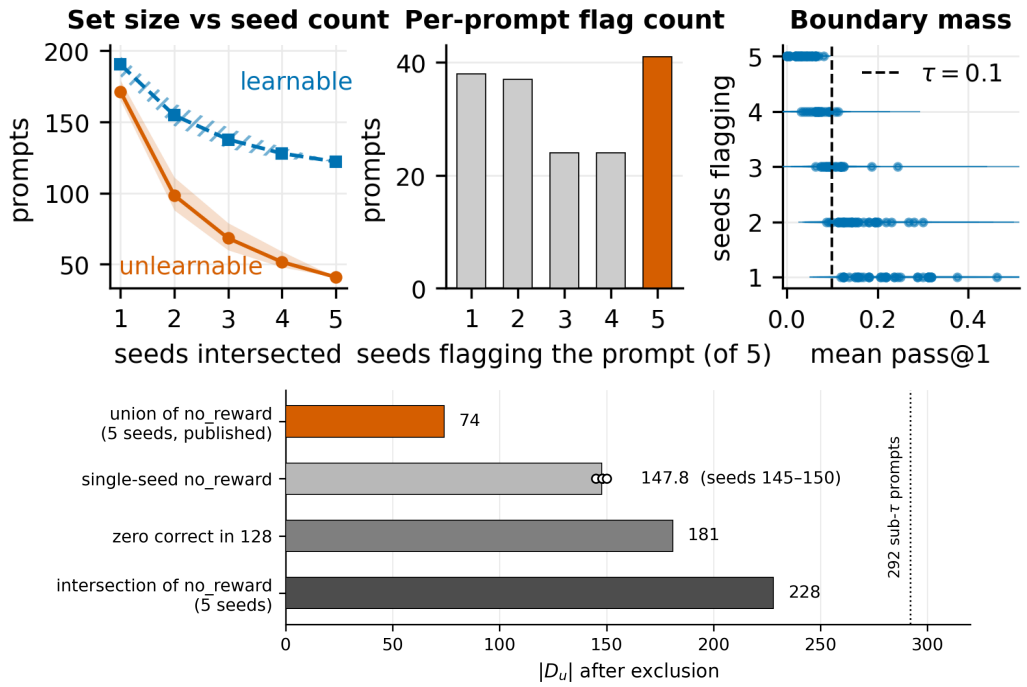


Figure 2: **The set is unstable in both seed count and aggregation operator.** Top: set size changes as more seeds are combined; many prompts are flagged by only a subset of seeds, with disagreement concentrated near $\tau = 0.1$. Bottom: holding the same 292 sub-threshold candidates fixed, the published aggregation rule applied across our five runs returns 74 prompts, individual seeds return 145–150 (mean 147.8), a zero-correct-in-128 rule returns 181, and an alternative aggregation over the same five runs returns 228.

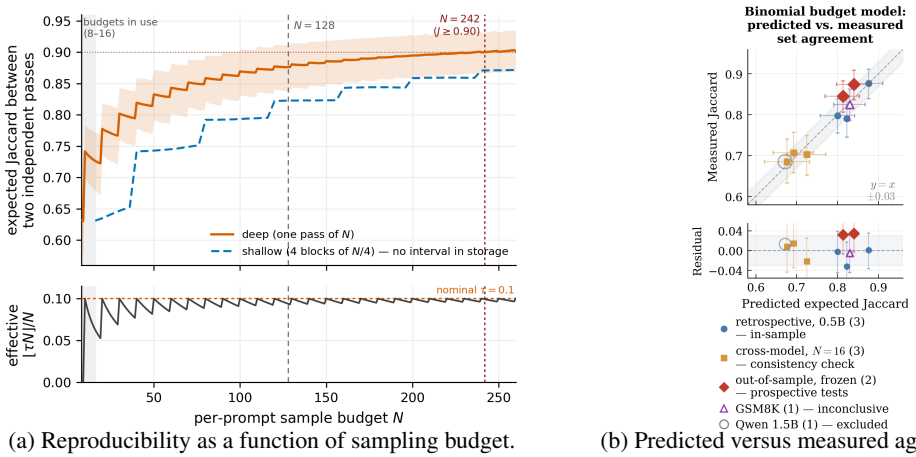
Is the disagreement mainly training variation? To remove training variation entirely, we evaluate the same checkpoint twice and change only the sampled rollouts. At $N = 32$, the two evaluations of the same model have Jaccard 0.798, compared with 0.751 across independently trained seeds. Expressed as disagreement, resampling one fixed model reproduces approximately 81% of the apparent cross-seed effect. The same qualitative result remains at $N = 128$. Different training runs matter, but they are not required to generate most of the membership instability.

Could our shorter generation cap be creating the set movement? We re-evaluate all 1,023 prompts at $N = 128$ with a 5,120-token cap, changing nothing else. The 1,024- and 5,120-token evaluations agree at Jaccard $0.8984\,[0.8639, 0.9310]$, while two independent passes at the same budget agree at $0.8770\,[0.8397, 0.9127]$. Truncation falls from 0.0961 to 0.0320, but the movement in set membership remains within the ordinary repeatability scale. This control supports the evaluation-side instability result; it does not recreate training under the original 5,120-token configuration.

3.4 How much evaluation does a difficulty-defined set need?

The previous section identifies two different problems: the published aggregation operator changes the set it selects, and even a well-defined fixed-τ set is noisy when $p(x)$ is estimated from few rollouts. We now address the constructive question: if we want to estimate D_τ, how much evaluation is required?

At equal cost, should the budget be spent deeply or split into shallow passes? We compare two ways of using the same budget of 128 rollouts per prompt. In the first, all 128 responses are used together to estimate the prompt's pass rate and apply the difficulty threshold once. In the second, the 128 responses are split into four independent blocks of 32, and a prompt is retained only if all four blocks classify it as difficult. This gives a direct comparison because both procedures apply the same



(a) Reproducibility as a function of sampling budget. (b) Predicted versus measured agreement.

Figure 3: **The sampling budget can be predicted before it is spent.** (a) Predicted Jaccard agreement between two independent threshold-defined sets as a function of per-prompt budget N; the lower panel shows the discrete effective threshold that produces the small-N sawtooth. (b) Predicted versus measured Jaccard. The filled diamonds are the two prospectively frozen $N = 40$ and $N = 64$ predictions. Retrospective and cross-model consistency checks are shown for context; the explicitly excluded 1.5B point is not treated as a validation result.

effective threshold:

$$\frac{\lfloor 0.1 \times 128 \rfloor}{128} = \frac{\lfloor 0.1 \times 32 \rfloor}{32} = 0.09375.$$

Using all 128 rollouts together gives a Jaccard of $0.8770\,[0.8397, 0.9127]$, while splitting them into four blocks gives $0.7906\,[0.7426, 0.8398]$. The difference is $+0.0864\,[+0.0380, +0.1349]$. Since both procedures use exactly the same number of rollouts, the improvement comes from how the budget is allocated rather than from additional compute. This result is specific to evaluation on a fixed checkpoint and does not imply that a single-pass rule is always optimal when training variation is also involved.

How many rollouts are required for a target level of reproducibility? For a prompt with true success probability p, evaluating it with N rollouts gives a random number of correct responses, K, following a binomial distribution. A prompt is labeled difficult when the observed number of successes falls below the threshold defined by τ. This gives $q_N(p)$, the probability that a prompt with success probability p receives the difficult label at a particular rollout budget. To use this for the full set of prompts, we need an estimate of how the underlying success probabilities are distributed across the dataset. We obtain this from the pooled $N = 128$ evaluation, while accounting for the fact that the observed pass rates are themselves noisy finite-sample estimates. We do this using the Kiefer–Wolfowitz nonparametric maximum-likelihood estimator (Kiefer and Wolfowitz, 1956). Once this distribution is estimated, we can predict how much two independent evaluations at the same rollout budget will agree with each other. This gives the predicted Jaccard agreement for any choice of N, and allows us to determine how many rollouts are needed to reach a chosen level of reproducibility.

Figure 3a shows how this agreement improves as the evaluation budget increases. With only 8 rollouts per prompt, the predicted Jaccard agreement is 0.6322; with 16 rollouts it rises to 0.7265, and with 128 rollouts it reaches 0.8759. We can then ask for the rollout budget required to maintain a particular level of agreement. In our setting, reaching a stable agreement of 0.80 requires about 40 rollouts per prompt, while 0.90 and 0.95 require projected budgets of 242 and 2,111, both beyond our $N = 128$ fitting range. We therefore treat these as projections and do not recommend acting on the 0.90 and 0.95 budgets without direct measurement above $N = 128$. These values are specific to this model, dataset, and threshold, because the required budget depends strongly on how many prompts lie close to the decision boundary. The important contribution is therefore not the particular number of rollouts, but the procedure for calculating it. The curve also has a small sawtooth pattern at low budgets because the threshold can only correspond to an integer number of successful responses. As N increases, sampling noise decreases, but the effective cutoff occasionally shifts at the same time.

This means that adding a few more rollouts can sometimes reduce agreement locally. For that reason, we report the first budget from which the desired agreement remains satisfied, rather than an isolated point where the curve happens to cross the target.

Does the budget model predict measurements that were not used to fit it? We first perform six retrospective and cross-model consistency checks, including checks on a different model family (Llama-3.2-3B); a further Qwen2.5-1.5B row is reported but excluded as a diagnostic. We then freeze two predictions for budgets that had not yet been measured and generate the corresponding evaluations only afterward. For $N = 40$, the frozen prediction is $0.8131\,[0.7699, 0.8533]$ and the measured Jaccard is $0.8450\,[0.8063, 0.8833]$. For $N = 64$, the prediction is $0.8397\,[0.7980, 0.8792]$ and the measurement is $0.8742\,[0.8369, 0.9091]$. Both lie inside their pre-specified intervals (Figure 3b). A homogeneous-pass-rate null can match the marginal flagged fraction by construction but misses observed Jaccard by 0.19–0.53, whereas the distributional model misses the corresponding retrospective measurements by only 0.008–0.032. Reproducibility is therefore controlled by the distribution of prompt probabilities around the threshold, not merely by the final set size.

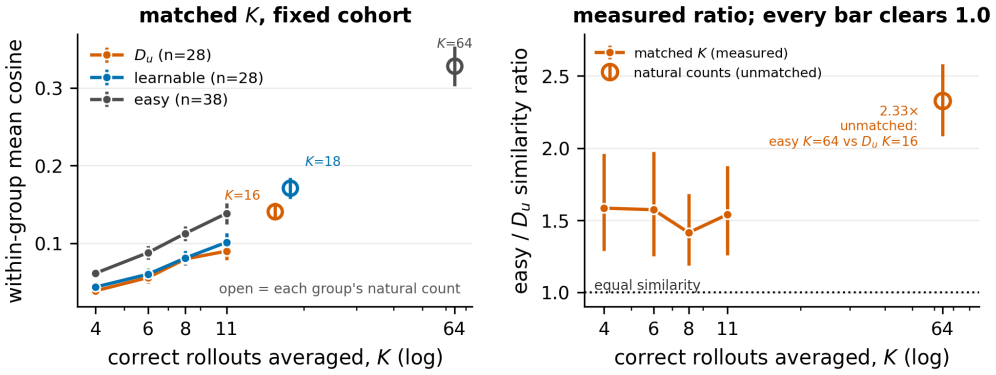


Figure 4: **Gradient similarity after matching estimator sample count.** Left: within-group mean cosine as the number K of correct rollouts averaged per prompt is matched across cohorts. The fixed matched cohort contains 28 D_u, 28 learnable, and 38 easy prompts. Right: the easy-to-D_u similarity ratio. Matching reduces the unmatched 2.327 times ratio to approximately 1.5 times, while every tested matched level remains above one. Open markers show each group's natural, unmatched sample count. At $K = 11$, matching retains only 28 of the 71 D_u prompts in the gradient cohort; the matched values therefore demonstrate sensitivity to unequal estimator sample count in an eligible subset, rather than providing corrected full-cohort estimates.

3.5 Does the gradient gap persist after matching sample count?

The proposed explanation for unlearnability relies partly on the observation that gradients from D_u prompts are less similar to one another than gradients from easy prompts. However, the two groups are not measured under the same conditions. Each prompt-level gradient is estimated by averaging the gradients of its correct rollouts, and difficult prompts naturally produce fewer correct responses. In the $N = 200$ gradient cohort, D_u prompts have a median of only 8 correct rollouts, compared with 65 for easy prompts. Their gradient estimates are therefore based on substantially fewer samples, creating a separate measurement problem from the instability of the difficulty labels.

To test how much this difference in sample count affects the result, we recompute the comparison using the same number of correct rollouts for every prompt. In the original unmatched analysis, the easy-to-D_u similarity ratio is 2.327 times. When both groups are restricted to $K = 11$ correct rollouts per prompt, the ratio falls to 1.539 times, with a 95% confidence interval of $[1.26, 1.88]$ (Figure 4). The same pattern appears at $K = 4, 6, 8$, and 11: every matched estimate is smaller than the original ratio, while all remain above one. Matching the sample count therefore weakens the reported gradient separation substantially, but does not eliminate it.

This comparison also introduces an important selection effect. Across $K \in \{4, 6, 8, 11\}$, eligible D_u retention moves from $55/71$ (77.5%, ratio $1.584\times$) at $K = 4$ to $47/71$ ($1.572\times$) at $K = 6$,

$38/71$ ($1.413\times$) at $K = 8$, and $28/71$ (39.4%, $1.539\times$) at $K = 11$, alongside 38 easy prompts in the fixed cohort. The retained D_u prompts are necessarily those with more successful responses and are therefore easier than many prompts removed by matching. On a log scale, matching removes 48.9% ($[29.3, 71.2]\%$) of the observed separation, which we interpret not as a causal decomposition or full-cohort correction, but as evidence that unequal sample count contributes substantially to the original gradient gap (Appendix C).

We next consider whether the remaining slow-learning gap can be explained by how often difficult prompts contribute to GRPO updates. With $G = 8$, a prompt produces no update when all sampled rewards are identical, so difficult prompts have fewer opportunities to contribute. Across the published partitions of Chen et al. (2026) (156 unlearnable, 159 learnable, and 586 easy prompts), unlearnable prompts receive an average of 2.14 contributing steps per run, compared with 3.13 for learnable prompts and 5.26 for easy prompts. This difference is substantial, but it does not explain the phenomenon on its own. None of the 156 prompts in this published unlearnable cohort is completely starved across all five runs, and 35 of them, or 22.4%, receive at least the mean exposure of the learnable group while still improving slowly.

These results rule out the simple form of exposure starvation in which D_u prompts fail to learn because they receive no usable update opportunities. They do not rule out a stronger possibility in which the rewarded trajectories themselves contain weak or uninformative learning signal, since outcome-equivalent trajectories can still differ in reasoning quality (Mei et al., 2026). We cannot test that explanation directly because the training-time response strings were not stored. We therefore distinguish between *exposure starvation*, which is not supported by our measurements, and *content starvation*, which remains unresolved.

A second possibility is that GRPO's reward-standard-deviation normalization systematically suppresses updates on hard prompts. If this were the case, D_u prompts should receive smaller update scales on the steps where they contribute. We observe the opposite. Conditional on a prompt producing non-zero reward variance, the mean $1/\sigma$ scale is 2.8360 for D_u prompts and 2.4262 for easy prompts, about 17% larger for the harder group. The main asymmetry is therefore in how often a usable update is formed, not in a smaller normalization factor once an update exists. Together, these results substantially weaken the original gradient-based explanation and rule out two simple optimization explanations in the forms we can directly measure, while leaving the underlying cause of the slow-learning effect unresolved.

3.6 An unstable difficulty rule does not necessarily imply unstable training

The results above concern the reliability of a difficulty-defined set and the scientific claims made from it, but a separate question is whether an unstable difficulty rule necessarily makes difficulty-aware RLVR ineffective. We test this using two neighboring constructions that remove very different amounts of usable training signal: after accounting for GRPO's zero-variance filter, one removes approximately 5.0% of the usable training signal while the other removes 27.6%, a difference of more than fivefold. We train five seeds under each construction and compare them with size-matched random-removal controls. Despite this large difference in removed signal, the resulting models differ by less than one held-out accuracy point. The registered sensitivity of the experiment is not sufficient to resolve effects of that size, so this should not be interpreted as evidence of equivalence. It does, however, establish an important boundary on our claim: a large change in the reported set does not produce an equally large change in training performance in this setting. Our strongest conclusion is therefore about measurement and inference: our experiments do not show that every training method using a noisy difficulty signal must fail.

4 Conclusion

We revisited the recently reported unlearnability phenomenon in RLVR and found a more nuanced result. The affected prompts improve at roughly one third of the learnable rate rather than not at all, but the prompts used to define them are far less stable than the effect itself. The published aggregation rule does not consistently recover a fixed-threshold set, and finite-sample evaluation explains much of the apparent disagreement across training seeds. We provide a way to determine how much evaluation is required for reproducible difficulty assignment and validate its predictions at previously unmeasured budgets. We also show that the gradient-similarity difference proposed

to explain unlearnability shrinks substantially when estimator sample count is matched, although a residual gap remains. The main lesson is therefore not that unlearnability disappears, but that low-budget difficulty labels are unreliable for classifying individual prompts as "unlearnable," or for supporting mechanistic claims built on that classification.

The same uncertainty estimates also apply on the training side rather than only to evaluation budgets. Under binary rewards and group size G, a prompt yields a non-degenerate GRPO group with probability $c_G(p) = 1 - p^G - (1-p)^G$, so a posterior over p gives the expected contribution of the next group and supports allocating rollouts to where an update is most likely to form; we regard this as the natural extension of our framework.

Our study has two main limitations. Our training results come from a single model and prompt distribution (Qwen2.5-0.5B on MATH), and our configuration differs from the original in training and evaluation response caps, epochs, and dataset subset (Appendix E.1); our evaluation-side controls therefore do not establish how these findings behave under the original training configuration. Validating them there, and across further models, datasets, and RLVR algorithms, is the clearest next step.

References

Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron C Courville, and Marc Bellemare. Deep reinforcement learning at the edge of the statistical precipice. *Advances in neural information processing systems*, 34:29304–29320, 2021.

Priyank Agrawal, Ankur Samanta, Shervin Ghasemlou, Jalaj Bhandari, Kavosh Asadi, Daniel Jiang, and Aditya Modi. Off-context grpo: Learning to reason on hard problems using privileged information. *arXiv preprint arXiv:2607.19313*, 2026.

Sanghwan Bae, Jiwoo Hong, Min Young Lee, Hanbyul Kim, JeongYeon Nam, and Donghyun Kwak. Online difficulty filtering for reasoning oriented reinforcement learning. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 700–719, 2026.

Andrei Baroian and Rutger Berger. Prompt replay: speeding up grpo with on-policy reuse of high-signal prompts. *arXiv preprint arXiv:2603.21177*, 2026.

Vladislav Beliaev. Max out grpo signal: Adaptive trace prefix control for hard reasoning problems. *arXiv preprint arXiv:2607.07674*, 2026.

Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, Assya Trofimov, Brennan Nichyporuk, Justin Szeto, Nazanin Mohammadi Sepahvand, Edward Raff, Kanika Madan, Vikram Voleti, et al. Accounting for variance in machine learning benchmarks. *Proceedings of machine learning and systems*, 3:747–769, 2021.

Yulin Chen, He He, and Chen Zhao. The unlearnability phenomenon in rlvr for language models. *arXiv preprint arXiv:2605.16787*, 2026.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Ganqu Cui, Yuchen Zhang, Jiacheng Chen, Lifan Yuan, Zhi Wang, Yuxin Zuo, Haozhan Li, Yuchen Fan, Huayu Chen, Weize Chen, et al. The entropy mechanism of reinforcement learning for reasoning language models. *arXiv preprint arXiv:2505.22617*, 2025.

Yanqi Dai, Yuxiang Ji, Xiao Zhang, Yong Wang, Xiangxiang Chu, and Zhiwu Lu. Harder is better: Boosting mathematical reasoning via difficulty-aware grpo and multi-aspect question reformulation. *arXiv preprint arXiv:2601.20614*, 2026.

Jesse Dodge, Suchin Gururangan, Dallas Card, Roy Schwartz, and Noah A Smith. Show your work: Improved reporting of experimental results. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2185–2194, 2019.

Xiaoliang Fu, Jiaye Lin, Yangyi Fang, Binbin Zheng, Chaowen Hu, Zekai Shao, Cong Qin, Lu Pan, Ke Zeng, and Xunliang Cai. Maspo: Unifying gradient utilization, probability mass, and signal reliability for robust and sample-efficient llm reasoning. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 41348–41365, 2026.

Zhaolin Gao, Joongwon Kim, Wen Sun, Thorsten Joachims, Sid Wang, Richard Yuanzhe Pang, and Liang Tan. Prompt curriculum learning for efficient llm post-training. In *International Conference on Learning Representations*, volume 2026, pages 58614–58652, 2026.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Zhezheng Hao, Hong Wang, Haoyang Liu, Jian Luo, Jiarui Yu, Hande Dong, Qiang Lin, Can Wang, and Jiawei Chen. Rethinking entropy interventions in rlvr: An entropy change perspective. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 31105–31133, 2026.

Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.

Yuhua Jiang, Jiawei Huang, Yufeng Yuan, Xin Mao, Qianchuan Zhao, and Lin Yan. Risk-sensitive reinforcement learning for alleviating exploration dilemmas in large language models. In *International Conference on Learning Representations*, volume 2026, pages 47117–47141, 2026.

Amirhossein Kazemnejad, Milad Aghajohari, Eva Portelance, Alessandro Sordoni, Siva Reddy, Aaron Courville, and Nicolas Le Roux. VinePPO: Refining credit assignment in RL training of LLMs. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu, editors, *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 29557–29590. PMLR, 13–19 Jul 2025. URL `https://proceedings.mlr.press/v267/kazemnejad25a.html`.

Jack Kiefer and Jacob Wolfowitz. Consistency of the maximum likelihood estimator in the presence of infinitely many incidental parameters. *The Annals of Mathematical Statistics*, 27(4):887–906, 1956.

Pang Wei Koh and Percy Liang. Understanding black-box predictions via influence functions. In *International conference on machine learning*, pages 1885–1894. Pmlr, 2017.

Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, et al. Tulu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2024.

Jinghan Li, Junfeng Fang, Jinda Lu, Yuan Wang, Xiaoyan Guo, Tianyu Zhang, Xiang Wang, and Xiangnan He. Enhancing multi-modal llms reasoning via difficulty-aware group normalization. *arXiv preprint arXiv:2602.21743*, 2026.

Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *International Conference on Learning Representations*, volume 2024, pages 39578–39601, 2024.

Mingjie Liu, Shizhe Diao, Ximing Lu, Jian Hu, Xin Dong, Yejin Choi, Jan Kautz, and Yi Dong. Prorl: Prolonged reinforcement learning expands reasoning boundaries in large language models. *Advances in Neural Information Processing Systems*, 38:17998–18031, 2026.

Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding r1-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.

Haisen Luo, Yiwei Liu, Haoning Wang, Dan Liu, Junxi Yin, Haotian Wang, Lei Zhang, Xiaoyu Tian, Shuaiting Chen, Yuansheng Song, et al. Drift: Difficulty routing self-distillation with rhythm-gated exploration and success buffer training. *arXiv preprint arXiv:2606.30345*, 2026.

Tiehua Mei, Minxuan Lv, Leiyu Pan, Zhenpeng Su, Hongru Hou, Hengrui Chen, Ao Xu, and Deqing Yang. Good reasoning makes good demonstrations: Implicit reasoning quality supervision via in-context reinforcement learning. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 30452–30469, 2026.

Yujuan Pang, Jiaxin Li, Xin Sheng, Ran Peng, and Yong Ma. Beyond variance: Prompt-efficient rlvr via rare-event amplification and bidirectional pairing. *arXiv preprint arXiv:2602.03452*, 2026.

Garima Pruthi, Frederick Liu, Satyen Kale, and Mukund Sundararajan. Estimating training data influence by tracing gradient descent. *Advances in neural information processing systems*, 33: 19920–19930, 2020.

Yun Qu, Qi Wang, Yixiu Mao, Heming Zou, Yuhang Jiang, Weijie Liu, Clive Bai, Kai Yang, Yangkun Chen, Saiyong Yang, et al. Small generalizable prompt predictive models can steer efficient rl post-training of large reasoning models. *arXiv preprint arXiv:2602.01970*, 2026.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Xinyu Tang, Zhenduo Zhang, Yurou Liu, Xin Zhao, Zhiqiang Zhang, JUN ZHOU, et al. Towards high data efficiency in reinforcement learning with verifiable reward. In *International Conference on Learning Representations*, volume 2026, pages 32611–32635, 2026.

Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. Solving math word problems with process-and outcome-based feedback. *arXiv preprint arXiv:2211.14275*, 2022.

Shenzhi Wang, Le Yu, Chang Gao, Chujie Zheng, Shixuan Liu, Rui Lu, Kai Dang, Xiong-Hui Chen, Jianxin Yang, Zhenru Zhang, Yuqiong Liu, An Yang, Andrew Zhao, Yang Yue, Shiji Song, Bowen Yu, Gao Huang, and Junyang Lin. Beyond the 80/20 rule: High-entropy minority tokens drive effective reinforcement learning for llm reasoning. In D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen, editors, *Advances in Neural Information Processing Systems*, volume 38, Main Conference, pages 115452–115486. Curran Associates, Inc., 2025a. doi: 10.52202/085713-3850. URL `https://proceedings.neurips.cc/paper_files/paper/2025/file/a797c2d2e0c1fdabf4d1ab8cd0b465c6-Paper-Conference.pdf`.

Yiping Wang, Qing Yang, Zhiyuan Zeng, Liliang Ren, Liyuan Liu, Baolin Peng, Hao Cheng, Xuehai He, Kuan Wang, Jianfeng Gao, Weizhu Chen, Shuohang Wang, Simon Du, and yelong shen. Reinforcement learning for reasoning in large language models with one training example. In D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen, editors, *Advances in Neural Information Processing Systems*, volume 38, Main Conference, pages 122721–122764. Curran Associates, Inc., 2025b. doi: 10.52202/085713-4093. URL `https://proceedings.neurips.cc/paper_files/paper/2025/file/b1ea3f93167c55f3fc2999b68a170ff3-Paper-Conference.pdf`.

Xumeng Wen, Zihan Liu, Shun Zheng, Shengyu Ye, Zhirong Wu, Yang Wang, Zhijian Xu, Xiao Liang, Junjie Li, Ziming Miao, et al. Reinforcement learning with verifiable rewards implicitly incentivizes correct reasoning in base llms. In *International Conference on Learning Representations*, volume 2026, pages 49450–49483, 2026.

An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

Chenlu Ye, Zhou Yu, Ziji Zhang, Hao Chen, Narayanan Sadagopan, Jing Huang, Tong Zhang, and Anurag Beniwal. Beyond correctness: Harmonizing process and outcome rewards through rl training. *arXiv preprint arXiv:2509.03403*, 2025.

Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, juncai liu, LingJun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Ru Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Yonghui Wu, and Mingxuan Wang. Dapo: An open-source llm reinforcement learning system at scale. In *Advances in Neural Information Processing Systems*, volume 38, Main Conference, 2025. doi: 10.52202/085713-3775. URL `https://proceedings.neurips.cc/paper_files/paper/2025/hash/a4277440d50f1f15d2cb4c14f7e0c0d2-Abstract-Conference.html`.

Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in llms beyond the base model? In D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen, editors, *Advances in Neural Information Processing Systems*, volume 38, Main Conference, pages 57654–57689. Curran Associates, Inc., 2025. doi: 10.52202/085713-1933. URL `https://proceedings.neurips.cc/paper_files/paper/2025/file/537d5aa768c2d534016a4d06f87bc8fb-Paper-Conference.pdf`.

Erle Zhu, Dazhi Jiang, Yuan Wang, Xujun Li, Jiale Cheng, Yuxian Gu, Yilin Niu, Aohan Zeng, Jie Tang, Minlie Huang, et al. Data efficient rlvr via off-policy influence guidance. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 46167–46192, 2026.

Appendix — Contents and Index

Appendix A Reproduction and set construction in full . . . . . p. 15
A.1 The slow-learning effect . . . . . p. 15
A.2 How the published construction changes the selected set . . . . . p. 15
A.3 Evaluation noise versus training-seed variation . . . . . p. 16

Appendix B Sampling-budget model and validation . . . . . p. 16
B.1 From success probability to reproducibility . . . . . p. 16
B.2 Equal-cost allocation of the rollout budget . . . . . p. 18
B.3 Validation of the sampling-budget model . . . . . p. 18
B.4 Discrete thresholds and what transfers across settings . . . . . p. 20

Appendix C Gradient and downstream sensitivity analyses . . . . . p. 21
C.1 Propagation of set uncertainty to gradient similarity . . . . . p. 21
C.2 Matched-gradient analysis and cohort composition . . . . . p. 21

Appendix D Optimization diagnostics and training impact . . . . . p. 22
D.1 Exposure, zero-variance filtering, and GRPO normalization . . . . . p. 22
D.2 Does difficulty construction materially change training? . . . . . p. 23

Appendix E Experimental configuration and replication audit . . . . . p. 24
E.1 Experimental configuration and statistical procedures . . . . . p. 24
E.2 Generation length and answer extraction . . . . . p. 25

A Reproduction and set construction in full

A.1 The slow-learning effect

We first reproduce the training behavior using the easy ($n = 586$), learnable ($n = 159$), and unlearnable ($n = 156$) partitions published by Chen et al. (2026). The groups remain clearly separated throughout training. In our primary run (Seed 1), estimated within-prompt reward slopes per 100 optimizer steps are $+0.3800\,[+0.3599, +0.4006]$ (easy), $+0.1119\,[+0.0879, +0.1368]$ (learnable), and

$$+0.0422\,[+0.0259, +0.0590] \quad (D_u).$$

The D_u interval excludes zero, and the positive slope replicates across all five independent training seeds with all 156 prompts retained in every seed: $+0.0422$, $+0.0349$, $+0.0266$, $+0.0321$, and $+0.0280$ per 100 steps (five-seed mean $+0.0328$, compared with $+0.1104$ for learnable and $+0.3669$ for easy). Fixed-checkpoint evaluations of the same 156 prompts from the initial policy to step 120 ($N = 256$) likewise confirm positive pass-rate gains in every seed ($+0.1232$ to $+0.1306$).

We rely on the within-prompt slope across steps 1–120 rather than a narrow terminal reward window for a sampling reason specific to GRPO minibatch collection. In our trainer (`ray_trainer.py`), reward statistics are logged to `prompt_reward_stats.csv` for every generated group of $G = 8$ rollouts *prior* to zero-variance filtering (`processed=False`), so all $0/8$ and $8/8$ groups are recorded and no prompts are dropped from the dataset across epochs. However, each optimizer step draws prompts until 32 non-zero-variance groups are accumulated (giving an average of 6.68 appearances per prompt across the 120-step run). Consequently, a narrow 10-step terminal window (steps 111–120) visits only a subset of each cohort (60 of 156 D_u, 72 of 159 learnable, and 181 of 586 easy prompts, where the learnable-minus-D_u terminal level gap is $+0.0754\,[+0.0279, +0.1246]$). By contrast, fitting an ordinary least-squares linear trend within each individual prompt across all of its unfiltered appearances over steps 1–120 (requiring at least 3 appearances) and averaging those per-prompt slopes across the fixed cohort includes **all 156 of 156 D_u, 159 of 159 learnable, and 586 of 586 easy prompts (zero prompts dropped)**.

The reproduction therefore supports a persistent difference in trainability on a fixed prompt set, rather than literal non-learning. Throughout the paper, D_u refers to the set produced by the published construction rather than to a claim that every member is intrinsically unlearnable.

A.2 How the published construction changes the selected set

Let $p(x)$ denote the true success probability of prompt x under a fixed policy. A fixed-threshold difficulty set,

$$D_\tau = \{x : p(x) \le \tau\},$$

is well defined. The published construction does not observe $p(x)$ directly. It first thresholds finite-sample success counts within individual runs and then combines those binary decisions across seeds. This distinction produces two separate sources of instability. First, prompts near τ can cross the threshold under repeated sampling even when the policy does not change. Second, applying intersections and exclusions across these noisy binary labels changes the set being selected as more runs are added. Holding the same 292 prompts identified as sub-threshold in a pooled $N = 128$ evaluation fixed, the published aggregation rule applied across our five runs returns 74 prompts. Individual seeds return between 145 and 150 prompts, with mean 147.8. A direct rule requiring zero successes in 128 evaluation samples returns 181, while an alternative aggregation over the same five runs returns 228. The difference caused by the aggregation operator is therefore much larger than the variation across individual training seeds.

The seed-count analysis shows the same behavior continuously rather than only at the two endpoint constructions. As more seeds are combined, the selected set shrinks instead of approaching a stable finite-τ set. Many prompts are flagged by only a subset of the five runs, and this disagreement is concentrated near $\tau = 0.1$. Under repeated independent application, an intersection retains only prompts whose probability of being flagged approaches one. For the binomial threshold rule considered here, this pushes the limit toward prompts with $p = 0$. The accompanying never-rewarded exclusion then removes those prompts as the number of opportunities to observe the exclusion condition grows. The resulting procedure therefore has a degenerate limiting behavior rather than consistently estimating D_τ.

The same preference for the lower tail is visible in a direct precision–recall comparison. Against the $N = 128$ reference set, the five-seed intersection has precision 0.976 and recall 0.541 $[0.421, 0.657]$. The construction therefore does not simply recover a cleaner version of the threshold-defined set. It preferentially keeps prompts far enough below the threshold to survive repeated noisy decisions. This behavior is important when interpreting intersections as a denoising operation. High precision alone can make the resulting set appear more reliable, but the corresponding loss in recall shows that the rule has changed the selected region of the difficulty spectrum. It acts as a selector for the lower tail rather than as a neutral estimator of the original fixed threshold. For prompts near the decision boundary, a more natural representation is therefore uncertainty in membership rather than a deterministic label. The sampling-budget analysis in Appendix B asks the related operational question: how many responses are required before that uncertainty becomes small enough for a binary label to reproduce reliably?

The published construction also removes prompts that never received positive reward during the training runs. In our data, this exclusion removes 218 prompts. A later $N = 128$ evaluation solves 116 of those prompts at least once. For this reason, we keep two constructions separate in the full analysis. The published observed-reward construction gives

$$|D_u^{\text{observed}}| = 74,$$

whereas defining the exclusion using demonstrated solvability under the deeper evaluation gives

$$|D_u^{\text{solvable}}| = 181.$$

The two sets overlap on 65 prompts. We use the 74-prompt set whenever reproducing the original claim, because that is the construction under audit, but we do not interpret the excluded prompts as demonstrated to be impossible for the model. The distinction is simple but important. "No positive reward was observed during these training runs" and "the policy cannot solve this prompt" are different statements. The former depends on the number of opportunities the prompt received and on the sampled trajectories; the latter is a property of the underlying policy and task. Together, these analyses separate three statements that would otherwise be easy to conflate: hard prompts exist, finite-sample estimates of difficulty can be unstable, and the particular multi-seed construction used to define D_u changes the set being selected. Our results challenge the last two without denying the first.

A.3 Evaluation noise versus training-seed variation

To determine how much of the disagreement actually comes from training, we repeat the difficulty assignment on the same fixed checkpoint and change only the sampled responses. At $N = 32$, two evaluations of the same checkpoint have Jaccard similarity 0.798. Across independently trained seeds, the corresponding similarity is 0.751. Expressed in terms of disagreement, the same-checkpoint experiment reproduces approximately 81% of the apparent cross-seed effect. At $N = 128$, the disagreement is 0.123 for repeated evaluation of the same model and 0.166 across seeds, corresponding to a similar share of approximately 74%. The $N = 32$ cross-seed quantity averages ten seed pairs, whereas the $N = 128$ result is based on a single pair, so we do not treat the two percentages as directly comparable estimates of one underlying constant.

A binomial model of the threshold decision predicts the same-model agreement to within 0.004. This supports the interpretation that finite evaluation sampling accounts for a large fraction of the observed membership instability.

Table 1 summarizes the main controls used to separate evaluation noise, generation-length effects, and sampling-budget allocation. The corresponding analyses are reported in Appendix A.3, Appendix E.2, and Appendix B.2, respectively.

B Sampling-budget model and validation

B.1 From success probability to reproducibility

The main paper treats difficulty assignment as a thresholding problem. For a prompt with latent success probability p, an evaluation with N sampled responses produces

$$K \mid p \sim \text{Binomial}(N, p),$$

Control	Comparison	Result	Interpretation
Same-model resampling	fixed checkpoint vs. cross-seed, $N = 32$	0.798 vs. 0.751	Most apparent cross-seed disagreement can arise from evaluation sampling alone.
Token-cap sensitivity	1,024 vs. 5,120 tokens	0.898 Jaccard	Set movement is no larger than the measured repeatability scale.
Equal-cost allocation	one $N = 128$ pass vs. 4×32	0.877 vs. 0.791	A deeper estimate is more reproducible at the same rollout cost.

Table 1: **Key robustness controls.** The first control separates evaluation noise from training variation; the second tests the generation-cap confound; the third asks how an equal sampling budget should be allocated.

and the prompt is assigned to the difficulty-defined set whenever

$$K \leq \lfloor \tau N \rfloor.$$

The corresponding probability that a prompt with success probability p is flagged is therefore

$$q_N(p) = \Pr(K \leq \lfloor \tau N \rfloor \mid p)\,.$$

This quantity makes the source of instability explicit. Prompts far below the threshold have $q_N(p)$ close to one, while prompts far above it have $q_N(p)$ close to zero. The uncertainty is concentrated near the threshold, where the same unchanged prompt can be included in one evaluation and excluded in another. The reproducibility of the final set therefore depends not only on N, but also on how much of the prompt distribution lies in this uncertain region.

We estimate that distribution from the pooled $N = 128$ evaluation. Directly using the empirical pass rates would treat their own sampling noise as genuine heterogeneity and would consequently overstate the stability of the thresholding procedure. We instead estimate a nonparametric mixing distribution over latent pass rates using the Kiefer–Wolfowitz maximum-likelihood formulation (Kiefer and Wolfowitz, 1956). The resulting distribution is used throughout the budget analysis; the raw empirical rates are not treated as noise-free estimates of p.

For two independent evaluations of the same prompt, the probability that it is included in both sets is $q_N(p)^2$, while the probability that it appears in their union is $2q_N(p) - q_N(p)^2$. Integrating these quantities over the estimated pass-rate distribution gives the expected intersection and union rates for two independent applications of the threshold rule. Their ratio gives the predicted population-level Jaccard agreement used throughout the paper. This analytic quantity is the ratio of expected membership rates, not the expectation of the finite-set Jaccard ratio itself,

$$J_N = \frac{\int q_N(p)^2 \, dF(p)}{\int \left[2q_N(p) - q_N(p)^2\right] \, dF(p)},$$

where F is the estimated distribution of latent prompt success probabilities. We evaluate this quantity over a dense integer grid of sampling budgets and invert the resulting curve to find the smallest budget from which a chosen reproducibility target remains satisfied.

The distinction between the first crossing and a stable crossing matters because the curve is not monotonic at small N. The threshold is implemented through an integer number of successes, so the effective cutoff is

$$\tau_{\text{eff}}(N) = \frac{\lfloor \tau N \rfloor}{N},$$

rather than exactly τ. Between successive values of N at which $\lfloor \tau N \rfloor$ increases, the effective threshold moves downward. Increasing N can therefore reduce sampling variance while simultaneously making the decision boundary stricter. The sawtooth pattern in the main paper is a direct consequence of this interaction, not Monte Carlo noise.

For the Qwen2.5-0.5B/MATH distribution at $\tau = 0.1$, the predicted Jaccard agreement values at representative budgets are:

N	8	16	32	64	128	256	512
Deep	0.6322	0.7265	0.7998	0.8413	0.8759	0.9022	0.9171

A stable target of 0.80 requires approximately $N = 40$, 0.90 requires $N = 242$, and 0.95 requires $N = 2111$. These values are properties of this particular threshold and pass-rate distribution; they are not universal recommendations. The transferable object is the calculation itself.

B.2 Equal-cost allocation of the rollout budget

The main paper compares two ways of spending the same evaluation budget. The **deep** rule uses all 128 responses to form one pass-rate estimate. The **shallow** rule divides those 128 responses into four disjoint blocks of 32 and retains a prompt only if all four blocks flag it. Both procedures therefore see exactly the same number of responses per prompt.

This comparison is interpretable only because their effective thresholds happen to match:

$$\frac{\lfloor 0.1 \times 128 \rfloor}{128} = \frac{\lfloor 0.1 \times 32 \rfloor}{32} = 0.09375.$$

This equality is specific to the chosen pair of budgets. In general, comparing one deep estimate with several shallow estimates can silently compare different effective thresholds, in which case any change in reproducibility mixes the effect of aggregation with a change in the underlying decision rule.

At equal cost, the deep rule is more reproducible. Its Jaccard across two independent evaluations is

$$0.8770\,[0.8397, 0.9127],$$

compared with

$$0.7906\,[0.7426, 0.8398]$$

for the four-block intersection. The paired difference is

$$+0.0864\,[+0.0380, +0.1349].$$

The same ordering remains after applying the never-rewarded exclusion. On the 759-prompt eligible subset, the difference grows to

$$+0.2138\,[+0.0905, +0.3354].$$

Figure 5 shows the complete equal-budget comparison. This experiment isolates evaluation noise on one fixed checkpoint; it is not a full replication of the published multi-seed construction, which also includes training variation. The result therefore supports a narrower conclusion: when estimating a threshold-defined set from a fixed policy, splitting a fixed budget into several thresholded blocks is less reproducible than using the same observations in one deeper estimate.

B.3 Validation of the sampling-budget model

Before making prospective predictions, we evaluated the model against several measurements that already existed for other parts of the study. The full retrospective and consistency results are shown in Table 2. The table contains seven rows. Six of them, the four Qwen2.5-0.5B rules and the two Llama-3.2-3B passes, are treated as retrospective or cross-model consistency checks. The seventh, Qwen2.5-1.5B, is shown as an excluded diagnostic and is not counted as validation evidence, for the reasons given below.

The first two deep evaluations are predicted to within 0.002. The largest discrepancy occurs for the four-block rule, where the model predicts 0.8228 against an observed 0.7906. Importantly, the model still recovers the ordering between deep and shallow evaluation, although it understates the measured gap: the predicted difference is 0.0531, compared with the observed 0.0864. The discrepancy is not explained by incorrect predicted set sizes, which match the observed sizes to within 1.8%, nor by obvious within-pass dependence: the observed block variance is 0.985 of the binomial expectation. We therefore retain the residual rather than fitting an additional correction to it. The cross-model rows should be read only as consistency checks. Each model's pass-rate distribution is estimated from the same $N = 32$ generation pass subsequently divided into two blocks, so these are not independent out-of-sample validations. The Qwen2.5-1.5B point is weaker still: 66.37% of its prompts are flagged and 79.44% of its rollouts are unparseable under our generation cap. In that setting, the measured set largely reflects the binding generation budget, and we do not count the point as validation evidence.

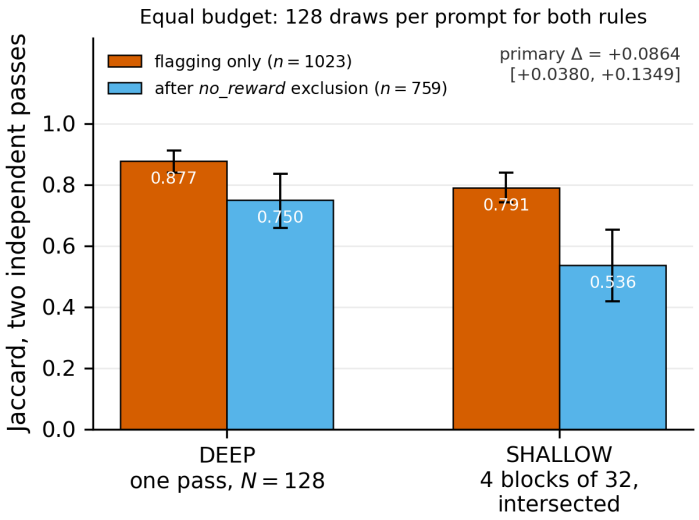


Figure 5: Equal-budget comparison of one deep $N = 128$ evaluation and four shallow blocks of 32. The deeper rule is more reproducible both before and after applying the never-rewarded exclusion. Error bars show 95% intervals.

Model / rule	Measured	Predicted	Error
Qwen2.5-0.5B, one pass $N = 32$	0.7977	0.8000	+0.0023
Qwen2.5-0.5B, one pass $N = 128$	0.8770	0.8759	-0.0011
Qwen2.5-0.5B, four blocks of 32	0.7906	0.8228	+0.0322
Qwen2.5-0.5B, two blocks of 16	0.7028	0.7248	+0.0220
Llama-3.2-3B, two blocks of 16	0.6844	0.6763	-0.0081
Llama-3.2-3B v2, two blocks of 16	0.7070	0.6926	-0.0144
Qwen2.5-1.5B, two blocks of 16	0.6849	0.6718	-0.0131

Table 2: Retrospective and cross-model consistency checks for the sampling-budget model. Six rows are treated as retrospective or consistency checks; the Qwen2.5-1.5B row is an excluded diagnostic and is not counted as validation evidence. The cross-model rows reuse the generation pass from which each pass-rate distribution was estimated and are therefore consistency checks rather than independent validation results.

A simpler model could reproduce the observed number of flagged prompts by assigning every prompt a single common success probability. Such a model deliberately removes the heterogeneity in prompt difficulty while preserving the marginal flagging rate. If set reproducibility were determined mainly by the final set size, this null should predict repeated-set agreement reasonably well. It does not. Across the retrospective comparisons, the homogeneous model misses measured Jaccard by approximately 0.19–0.53. The distributional model misses the four block-based comparisons in Table 2 by only 0.008–0.032, and the two single-pass deep evaluations by less than 0.003. Matching the marginal number of flagged prompts is therefore insufficient. What matters is how the latent success probabilities are distributed around the threshold: two prompt collections can produce the same set size while having very different probabilities of assigning the same individual prompts on a repeat evaluation. This comparison is important for interpreting the budget calculation. The model is not simply learning how many prompts should be flagged at a given N; it uses the shape of the pass-rate distribution to predict which membership decisions are likely to change.

The strongest validation in this study uses two sampling budgets that had not yet been measured. We froze and timestamped the predictions before generating the corresponding evaluation responses. The registered values were

$$N = 40 : \qquad J_{40} = 0.8131\,[0.7699, 0.8533],$$

and

$$N = 64: \qquad J_{64} = 0.8397\,[0.7980, 0.8792].$$

Only after these predictions were fixed did we generate the new evaluations, totaling 212,784 responses. The measured Jaccard values were

$$N = 40: \qquad 0.8450\,[0.8063, 0.8833],$$

and

$$N = 64: \qquad 0.8742\,[0.8369, 0.9091].$$

Both measurements fall inside their frozen prediction intervals. At $N = 40$, the complete measured interval also lies above the target reproducibility of 0.80, so the practical budget recommendation survives the prospective test rather than merely the point prediction. Both prospective residuals have nearly the same sign and magnitude: the model underpredicts reproducibility by roughly 0.03. We do not remove this offset after observing it. One possible contributor is that repeated evaluations share a deterministic verifier, which can create positive dependence not represented by a fully independent binomial calculation. This explanation was anticipated, but the retrospective checks contain residuals in both directions, so the available evidence does not establish it as the cause. For the practical budget calculation, the observed direction is conservative: underprediction recommends slightly more sampling rather than less. We also ran a prospectively frozen second-dataset check on the GSM8K test split (1,319 prompts at $N = 64$, 5,120-token cap), where the model predicted $0.8296\,[0.7904, 0.8672]$ and the measured Jaccard was $0.8246\,[0.7855, 0.8627]$ (point error $+0.0050$); because the measured confidence interval slightly straddled our pre-registered ± 0.0345 equivalence margin ($[0.7951, 0.8641]$), it is marked inconclusive under that stricter equivalence rule while its point estimate lies well inside the prediction interval. Figure 6 shows all retrospective, consistency, and prospective points together. The two frozen prospective MATH measurements are marked separately from the retrospective checks and the GSM8K test so that the figure does not visually overstate the amount of independent validation.

B.4 Discrete thresholds and what transfers across settings

The dense budget curve contains a feature that would disappear if only conventional powers of two were reported. Over small budgets, predicted reproducibility frequently decreases when N increases by one. A concrete example is

$$J_{10} = 0.7417, \qquad J_{19} = 0.7227,$$

so almost twice as many responses can correspond to a lower predicted agreement. The difference itself is small relative to the uncertainty of either estimate; the important observation is the systematic sawtooth pattern rather than this individual pair. The source is the discrete decision boundary. At $\tau = 0.1$, $N = 10$ permits one success while remaining at the threshold, whereas nearby values of N can implement a smaller effective cutoff until the next integer boundary is reached. The estimator therefore becomes statistically sharper at the same time that the classification rule changes. Over $N = 8$ to 160, this effect produces frequent local declines; beyond that range the sawtooth becomes much less prominent. This is precisely why the paper defines a required budget as the point from which a target remains satisfied, rather than the first value of N at which the curve crosses it.

The same issue also limits comparisons between aggregation rules. At very small block sizes, $\lfloor \tau B \rfloor = 0$, so a shallow rule becomes equivalent to requiring zero correct responses in every block. It is then no longer an alternative estimator of the same threshold used by the deeper rule. For $\tau = 0.1$, no useful deep-versus-shallow comparison can be formed from the available 32-draw screening passes without changing the effective threshold. The equality at 128 versus 4×32 is therefore a fortunate property of that particular comparison, not something that should be expected generally.

The numerical budget values in this paper should not be carried directly to another model or dataset. A model for which most prompts lie far from τ can obtain stable labels with relatively few responses, while another model with substantial probability mass near the same threshold may require much deeper evaluation. Changing τ can alter the required budget just as strongly. The same consideration applies during training as a model improves: a fixed number of responses need not provide the same reliability throughout the post-training trajectory. What does transfer is the procedure. Define the prompt-level quantity of interest, specify the threshold, estimate the distribution of latent success probabilities while accounting for finite-sample noise, and compute the predicted repeatability of the

resulting set before treating membership as fixed. If the available budget cannot achieve the required reproducibility, the uncertainty belongs in the downstream analysis rather than being removed by an arbitrary aggregation rule.

C Gradient and downstream sensitivity analyses

This appendix collects the analyses supporting Section 3.5. Figure 4 in the main paper remains the primary figure; the material here records the downstream propagation of set uncertainty and the cohort details behind the matched comparison.

C.1 Propagation of set uncertainty to gradient similarity

Low Jaccard is only scientifically important if uncertainty in membership is large enough to affect something computed on the selected population. We therefore redraw realizations of the threshold-defined set from the validated sampling model and recompute the easy-to-D_u gradient-similarity ratio. At $N = 8$, membership uncertainty alone contributes a full 95% interval width of 0.4128 to that ratio; at $N = 16$, the full width is 0.3090. These are 83% and 62%, respectively, of the full 95% sampling confidence-interval width already reported for the downstream statistic. At $N = 32$, the D_u-anchored and label-faithful variants fall on opposite sides of our pre-specified 0.25 materiality threshold, so we mark that budget unresolved. By $N = 128$, the additional full 95% interval width falls to 0.1375, and at $N = 200$ to 0.1161. Set uncertainty is therefore material at the small assignment budgets commonly used for difficulty labels and becomes much less important once the per-prompt measurement is sufficiently deep. Figure 7 shows the full budget-dependent propagation for both anchorings.

C.2 Matched-gradient analysis and cohort composition

The matched comparison in Figure 4 is computed on a fixed cohort of 28 D_u, 28 learnable, and 38 easy prompts intersected across six independent subsamples at each of $K \in \{4, 6, 8, 11\}$ correct rollouts averaged per prompt. Table 3 reports the within-group mean cosines, the easy-to-D_u similarity ratios with 95% bootstrap confidence intervals, and the per-K eligible prompt counts prior to cross-K intersection. At every tested level the easy-to-D_u similarity ratio lies below the unmatched value of $2.327\,[2.09, 2.59]$ and above one; the headline $K = 11$ comparison gives $1.539\,[1.26, 1.88]$, and the paired drift between $K = 4$ and $K = 11$ is $+0.044\,[-0.310, +0.413]$, which does not exclude zero.

Matching level	D_u **($n = 28$)**	**Learnable ($n = 28$)**	**Easy ($n = 38$)**	**Easy / D_u ratio [95% CI]**	**Per-K eligible (D_u / learn. / easy)**
$K = 4$	0.0389	0.0438	0.0616	$1.584\,[1.29, 1.96]$	$55/71,\ 35/40,\ 40/40$
$K = 6$	0.0560	0.0603	0.0880	$1.572\,[1.25, 1.97]$	$47/71,\ 33/40,\ 40/40$
$K = 8$	0.0798	0.0811	0.1128	$1.413\,[1.19, 1.68]$	$38/71,\ 30/40,\ 40/40$
$K = 11$	0.0900	0.1013	0.1385	$1.539\,[1.26, 1.88]$	$28/71,\ 28/40,\ 40/40$
Unmatched (fixed cohort)	0.1412 (K=16)	0.1711 (K=18)	0.3286 (K=64)	$2.327\,[2.09, 2.59]$	$28/71,\ 28/40,\ 38/40$
Unmatched (full $K \geq 4$)	0.1234 (n=55)	0.1639 (n=35)	0.3288 (n=40)	$2.666\,[2.38, 2.98]$	$55/71,\ 35/40,\ 40/40$

Table 3: Within-group mean gradient cosine similarities, easy-to-D_u ratios, and per-K eligibility across matching levels. The first five rows are evaluated on the fixed cross-K cohort ($28\,D_u, 28$ learnable, 38 easy, 6 subsamples per K); two of the 40 eligible easy prompts (`4422`, `6741`) are excluded from the fixed cohort because at least one $K = 4$ subsample draw produced zero score variance after verifier-guard filtering.

Matching introduces attrition. Requiring at least K guarded correct and K incorrect rollouts reduces D_u eligibility from $55/71$ at $K = 4$ (22.5% attrition) to $47/71$ at $K = 6$ (33.8%), $38/71$ at $K = 8$ (46.5%), and $28/71$ at $K = 11$ (60.6%). The retained D_u prompts are necessarily those that produced more successful responses, so they are easier than many of the prompts removed by the matching requirement.

This selection also explains the difference between two sample counts reported in the paper, which refer to different cohorts. Across the full $N = 200$ gradient cohort, D_u prompts have a median of 8 correct rollouts, compared with 65 for easy prompts. Within the fixed matched cohort, the natural, unmatched median counts shown by the open markers in Figure 4 are $K = 16$ for D_u, $K = 18$ for learnable, and $K = 64$ for easy prompts; the unmatched 2.327 times ratio compares easy prompts at $K = 64$ with D_u prompts at $K = 16$.

On a log scale, matching removes 48.9% of the observed separation on this fixed cohort, with a 95% interval of $[29.3, 71.2]\%$. We do not interpret this as a causal decomposition, because matching changes cohort membership: the matched ratio is evidence that unequal sample count contributes substantially to the original gradient gap, not a corrected estimate for the full D_u set.

Label-consistent cohort check. Because the $K = 11$ eligibility requirement retains D_u prompts with at least 11 correct rollouts out of 200, 11 of the 28 retained D_u prompts have initial-policy pass rates above $\tau = 0.1$. Restricting D_u to prompts whose initial-policy pass rate remains at or below $\tau = 0.1$ and easy prompts to $> \tau$ yields a label-consistent subset of 17 D_u and 37 easy prompts. On this subset, the unmatched easy-to-D_u ratio is $2.397\,[2.15, 2.66]$, the matched $K = 11$ ratio is $1.433\,[1.17, 1.78]$, and matching removes $58.8\%\,[38.5, 80.3]\%$ of the log separation (the paired difference in the matched $K = 11$ ratio relative to the 28/38 cohort is $-0.104\,[-0.271, +0.048]$).

5,120-token gradient sensitivity check. To test whether response truncation under the 1,024-token cap drives the matched-gradient result, we re-extracted $N = 200$ rollouts and per-prompt gradients at a 5,120-token cap on the common eligible support of 22 D_u, 25 learnable, and 40 easy prompts ($K = 11$, 3 subsamples). Increasing the cap from 1,024 to 5,120 tokens reduced incorrect-pool truncation from 28.89% to 20.17%, which did not clear our pre-registered truncation-halving gate ($< 14.45\%$) and widened the cross-group incorrect truncation spread from 0.0209 to 0.0783; this comparison therefore proceeded past a failed pre-registered gate and is reported solely as a sensitivity check rather than a cleaner measurement. On this common support, the matched $K = 11$ easy-to-D_u ratio is 1.546 at 1,024 tokens versus 1.444 at 5,120 tokens (per-subsample range 1.240 to 1.611), giving a 5,120-to-1,024 ratio of ratios of $0.934\,[0.714, 1.214]$. The evaluation-side generation-length control for set membership is reported in Appendix E.2.

D Optimization diagnostics and training impact

This appendix collects the diagnostics supporting the optimization analyses in Section 3.5 and the training experiment in Section 3.6.

D.1 Exposure, zero-variance filtering, and GRPO normalization

We analyze update exposure across the published partitions of Chen et al. (2026): 156 unlearnable, 159 learnable, and 586 easy prompts (901 partitioned prompts out of 1,023), tracked across all five 120-step GRPO training runs (≈ 6.68 appearances per prompt per run). With $G = 8$, a prompt contributes no GRPO update on a step where all eight sampled rewards are identical ($0/8$ or $8/8$). Across the five runs, the fraction of appearances discarded by this zero-variance filter is 68.2% for unlearnable prompts, 53.2% for learnable prompts, and 21.3% for easy prompts, corresponding to observed non-degenerate contributing fractions of $0.3180\,[0.2837, 0.3557]$, $0.4684\,[0.4353, 0.5017]$, and $0.7871\,[0.7729, 0.8009]$, respectively. These empirical rates closely track the binomial non-degeneracy prediction $c_8(\hat{p}) = 1 - \hat{p}^8 - (1 - \hat{p})^8$ computed from initial-policy pass rates (Pearson $r = +0.819$ across all 901 prompts).

In terms of optimizer exposure, unlearnable prompts receive a mean of $2.14\,[1.90, 2.39]$ contributing steps per run (median 1.80), compared with $3.13\,[2.91, 3.35]$ for learnable prompts (median 3.20; difference $+1.00\,[+0.66, +1.33]$ steps per run, or $1.47\times$) and $5.26\,[5.16, 5.36]$ for easy prompts (median 5.60). None of the 156 prompts in the published unlearnable cohort has zero contributing steps across all five runs (0.0%, versus 2.5% for learnable and 0.2% for easy), and 35 of the 156 prompts (22.4%) receive at least the mean exposure of the learnable group (≥ 3.13 steps per run) while still improving slowly. This rules out the simple form of exposure starvation in which unlearnable prompts fail to learn because they receive no usable update opportunities. It does not rule out content starvation, because the training-time response strings were not stored.

While zero-variance groups are discarded from the GRPO gradient update, they are still recorded in the unfiltered reward log (`processed=False`) used to estimate within-prompt learning slopes in Appendix A.1. The exposure asymmetry above therefore reflects how often each group supplies a non-zero gradient update during training, rather than missing observations in the reward-slope estimator.

A separate candidate explanation is that GRPO's reward-standard-deviation normalization suppresses updates on hard prompts on the steps where they do contribute. Conditional on a prompt producing non-zero reward variance, the mean $1/\sigma$ scale is $2.8360\,[2.8256, 2.8453]$ for D_u prompts, 2.6626 for learnable prompts, and $2.4262\,[2.4167, 2.4357]$ for easy prompts—a $1.169\times$ ($+16.9\%$) larger scale for the harder group because single-success ($1/8$) groups have the smallest positive Bernoulli standard deviation ($\sigma = \sqrt{7}/8 \approx 0.331$). A reduced normalization scale is therefore not supported as the explanation; the main asymmetry is in how often a usable update is formed.

D.2 Does difficulty construction materially change training?

To operationalize how much usable training signal a difficulty filter removes, let p_q denote prompt q's empirical pass@1 averaged across the five baseline seeds ($N = 32$). Under GRPO with group size $G = 8$, a prompt appearance produces a non-zero gradient update with probability $c_8(p_q) = 1 - p_q^8 - (1-p_q)^8$. Summing $c_8(p_q)$ over all 1,023 training prompts yields 516.3 expected contributing prompt-instances per epoch, which serves as the denominator for expected usable signal. Relative to this total, removing the 5-seed flagged intersection (218 prompts, 21.3% of prompts) removes 25.8 expected contributing instances per epoch, or 4.99% of usable training signal (because 83% of those 218 prompts have $p_q = 0$ and would have been discarded by GRPO's zero-variance filter anyway). Removing the 5-seed flagged union (404 prompts, 39.5% of prompts) removes 142.3 expected contributing instances per epoch, or 27.56% of usable signal—a $5.5\times$ larger reduction. Size-matched random removal controls remove 21.82% (random-218) and 39.87% (random-404) of expected usable signal.

We train five seeds (seeds 42–46) under each of the five arms for 120 optimizer steps at matched batch size and evaluate step-120 checkpoints ($N = 16$, greedy/temperature-0.6 guarded verifier) on two disjoint held-out benchmarks with zero overlap with the 1,023 training prompts: `e3_math500` (500 held-out MATH problems; pre-registered minimum detectable effect $\mathrm{MDE} = 0.91$ percentage points, empirical post-hoc $\mathrm{MDE} = 0.57$ points) and `e3_pool2000` (2,000 held-out problems from the remaining pool). Table 4 reports the complete results.

Training arm (5 seeds each)	**Prompts kept**	**Signal removed**	`e3_math500` **pass@1 (Δ vs. base)**	`e3_pool2000` **pass@1 (Δ vs. base)**
Arm 1: Full 1,023 baseline	1,023	0.00%	$31.40\% \pm 0.33$ (—)	$45.06\% \pm 0.14$ (—)
Arm 2: Drop 5-seed intersection	805	4.99%	$31.20\% \pm 0.42$ $(-0.21\,[-0.76, +0.34])$	$44.99\% \pm 0.14$ $(-0.06\,[-0.27, +0.14])$
Arm 4: Drop random-218 control	805	21.82%	$31.21\% \pm 0.34$ $(-0.19\,[-0.66, +0.28])$	$44.74\% \pm 0.56$ $(-0.32\,[-0.78, +0.14])$
Arm 3: Drop 5-seed union	619	27.56%	$30.95\% \pm 0.65$ $(-0.46\,[-1.17, +0.26])$	$44.98\% \pm 0.30$ $(-0.07\,[-0.36, +0.21])$
Arm 5: Drop random-404 control	619	39.87%	$30.83\% \pm 0.32$ $(-0.57\,[-0.97, -0.17])$	$44.48\% \pm 0.24$ $(-0.58\,[-0.93, -0.23])$

Table 4: Downstream held-out accuracy across five difficulty-removal arms (5 seeds per arm, 25 training runs total). All differences are in percentage points with 95% confidence intervals. Primary contrast (Arm 2 vs. Arm 3): $-0.25\,[-1.02, +0.52]$ points ($p = 0.4752$) on `e3_math500` and $-0.01\,[-0.30, +0.28]$ points ($p = 0.9558$) on `e3_pool2000`; size- and epoch-matched difference-in-differences $((\text{Arm } 3 - \text{Arm } 5) - (\text{Arm } 2 - \text{Arm } 4))$: $+0.13\,[-0.66, +0.92]$ and $+0.25\,[-0.31, +0.82]$ points.

Despite the $5.5\times$ difference in removed signal between the intersection and union constructions, the resulting models differ by only -0.25 points on `e3_math500` and -0.01 points on `e3_pool2000`. Because neither the pre-registered MDE (0.91 points) nor the empirical post-hoc MDE (0.57 points) is small enough to resolve sub-half-point effects, this result is not evidence of exact equivalence. We

also tested whether replacing noisy fixed-budget ($N = 32$) difficulty routing with sequential binomial-tail certification ($\delta = 0.05$, $N_{\min} = 8$, $N_{\max} = 64$) improves online difficulty-routed training (Prompt Replay (Baroian and Berger, 2026)): sequential certification reduces routing disagreement against a deep reference from 6.7% to 4.9% while cutting evaluation cost by 36.7% (from 32.0 to 20.3 rollouts per prompt), while downstream held-out accuracy remains invariant within < 0.8 points. Together, these experiments establish a clear boundary on the paper's claim: an unstable difficulty definition strongly perturbs prompt-level set membership and mechanistic attributions without necessarily producing a large change in aggregate held-out training accuracy.

E Experimental configuration and replication audit

E.1 Experimental configuration and statistical procedures

Our primary experiments use Qwen2.5-0.5B on the 1,023-prompt MATH training subset used throughout the study. RLVR training follows the GRPO setup of Chen et al. (2026), with $G = 8$ sampled responses per prompt and five independent training seeds. We use the original answer verifier without modification. Set-membership analyses are performed primarily with $N = 128$ evaluation rollouts per prompt, while the gradient analyses use $N = 200$ rollouts from the initial policy.

Existing assets. We use Qwen2.5-0.5B under the Apache 2.0 license, the MATH dataset under the MIT license, and the original unlearnability RLVR implementation under the Apache 2.0 license. All external assets are cited in the paper and used in accordance with their stated licenses and terms.

The experiments in this paper are a replication and extension rather than a byte-for-byte rerun of the original implementation. We therefore audited the effective configuration from the actual training and evaluation execution paths. The audit covers every shell variable passed to training and every data or sampling option forwarded to evaluation. At commit `1dce7fe`, we identified 47 effective constants: 30 match the original configuration and 17 differ.

Configuration constant	Ours	Original
Training		
Dataset name	`simplelr_qwen_level1to4_sub1k`	`simplelr_qwen_level1to4`
Maximum response length	1,024	5,120
Train batch size	32	256
PPO mini-batch size	16	64
PPO micro-batch size	2	32
Log-probability micro-batch size	4	128
Micro rollout batch size	128	1,024
Tensor parallelism	1	2
GPUs per node	1	4
Rollout GPU memory utilization	0.45	0.75
Total epochs	12	50
Save frequency	40	20
Test frequency	100,000	5
Validation batch size	200	1,000
Evaluation		
Test data	Training subset used here	Original test split
Maximum generation length	1,024	5,120
GPU memory utilization	0.60	0.75

Table 5: Configuration differences identified from the executed training and evaluation paths. The main text highlights differences that directly affect scientific interpretation; this table records the complete audit.

We divide the differences into four classes. **Direct-science** differences change the data or generation budget and can directly alter the measured quantities. **Optimization** differences modify the update protocol. **Execution-not-invariant** differences arise from adapting the original multi-GPU setup to a single-GPU execution path; we do not assume these changes are bitwise or stochastically invariant.

Operational differences affect logging, checkpointing, or evaluation frequency but do not directly define the training objective. Table 5 lists the complete set of differing constants; it does not annotate each row with its class.

The most important difference for the results in this paper is the generation budget. Our training and primary evaluations use a maximum response length of 1,024 tokens, whereas the original scripts default to 5,120. This difference affects hard prompts more strongly than easy prompts and therefore cannot be treated as an ordinary implementation detail. We address the evaluation-side consequence directly by repeating the full $N = 128$ evaluation with a 5,120-token budget. The training runs themselves remain at 1,024 tokens, so none of our training-dependent results should be interpreted as an exact numerical reproduction of the original configuration.

The dataset also differs from the original execution path: our experiments use the 1,023-prompt subset analyzed throughout this paper rather than the larger dataset referenced by the original baseline script. This choice fixes the set of prompts across the replication, deep evaluations, sampling-budget analysis, and gradient experiments. It also means that numerical quantities such as set size and required sampling budget are specific to this experimental setting.

Compute resources. All reported GPU experiments were run on a single NVIDIA GeForce RTX 3090 Ti with 24 GB of VRAM. Most training and evaluation runs took approximately 16–24 hours to complete, while lightweight post-processing and statistical analyses required substantially less compute.

Confidence intervals throughout the paper are obtained by non-parametric percentile bootstrap, with prompts resampled as the primary unit of independence, and are reported as 95% intervals in square brackets. Specifically: (i) within-prompt training reward slopes fit an ordinary least-squares line to each prompt's unfiltered step-level group means over steps 1–120 (requiring ≥ 3 appearances, satisfied by all 901 partitioned prompts) and resample prompts within each cohort with $5{,}000$ replicates (seed `20260815`); (ii) empirical Jaccard agreements resample the 1,023 prompts with $2{,}000$ replicates, while sampling-budget prediction intervals draw $6{,}000$ independent Monte Carlo realizations of pairs of thresholded evaluations from the 1,001-grid Kiefer–Wolfowitz NPMLE mixing distribution F (fitted on the pooled $N = 128$ counts via $3{,}000$ EM iterations to convergence tolerance 10^{-12}, seed `20260812`); and (iii) matched-K gradient ratios and log-gap reductions resample prompts within each cohort with $5{,}000$–$20{,}000$ replicates (seed `20260808`), sharing the resampled prompt draw across the natural-count and matched-K estimators on each bootstrap iteration so paired differences and log-gap shares preserve within-prompt covariance. Reward slopes are reported per 100 optimizer steps. The two prospective budget predictions in Appendix B.3 were frozen and timestamped, together with their intervals, before the corresponding evaluation responses were generated.

E.2 Generation length and answer extraction

The 1,024-token cap produces a substantial number of truncated responses. On the reference evaluation, increasing the budget from 1,024 to 5,120 tokens reduces the overall truncation rate from 0.0961 to 0.0320, and the answer-extraction failure rate from 0.0977 to 0.0329. Almost all extraction failures at the shorter budget are truncations rather than failures of the verifier itself.

Crucially, this correction does not substantially change the difficulty-defined set. The $N = 128$ sets obtained at 1,024 and 5,120 tokens have Jaccard similarity $0.8984\,[0.8639, 0.9310]$. Two independent evaluations at the same budget agree at $0.8770\,[0.8397, 0.9127]$. The movement produced by increasing the token budget is therefore no larger than the repeatability variation already present when the evaluation configuration is held fixed.

This control addresses only the evaluation-side set-membership result. Because our training trajectories were generated with the shorter limit, it does not establish that the training curves or gradient quantities would be numerically unchanged under the original 5,120-token configuration.

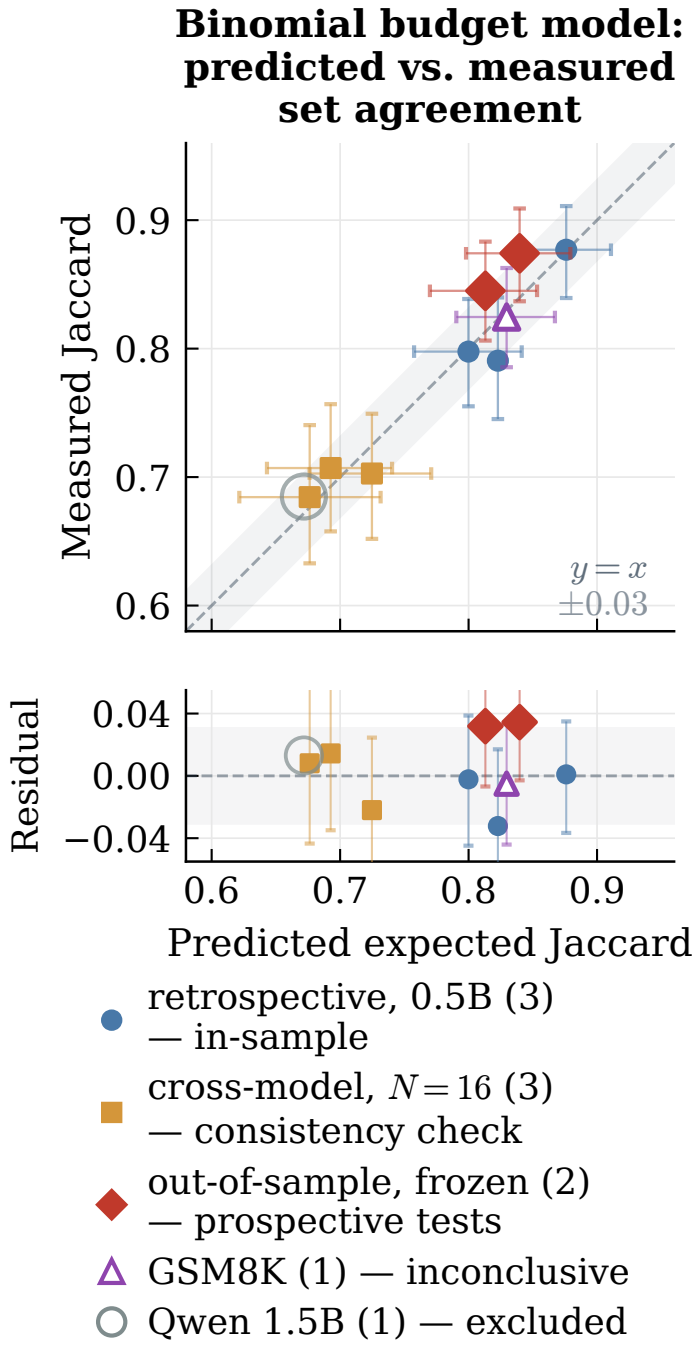


Figure 6: Predicted versus measured Jaccard agreement across retrospective checks, cross-model consistency checks, and prospectively frozen predictions. Filled diamonds mark the two prospective measurements; the Qwen2.5-1.5B point is shown separately and is not counted as validation evidence.

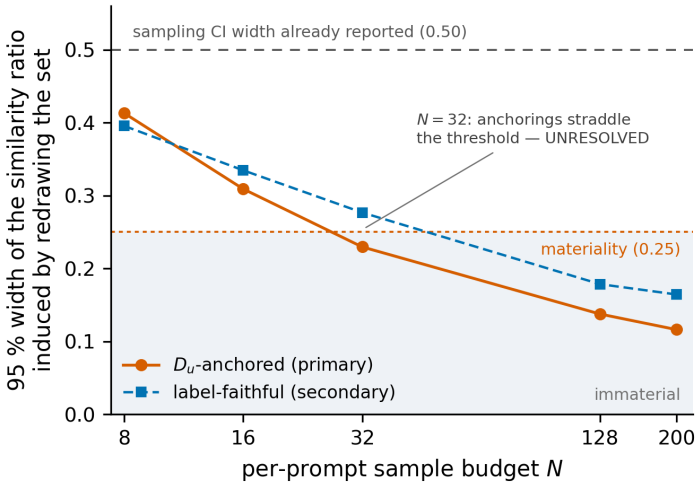


Figure 7: Full 95% interval width of the easy-to-D_u gradient-similarity ratio induced by redrawing the threshold-defined set from the validated sampling model, as a function of per-prompt budget N. The D_u-anchored variant is primary and the label-faithful variant secondary. The dotted line marks the pre-specified 0.25 materiality threshold; the dashed line marks the 0.50 sampling confidence-interval width already reported for the downstream statistic. At $N = 32$ the two anchorings straddle the threshold.